Color Block Tracking

1. Function Description

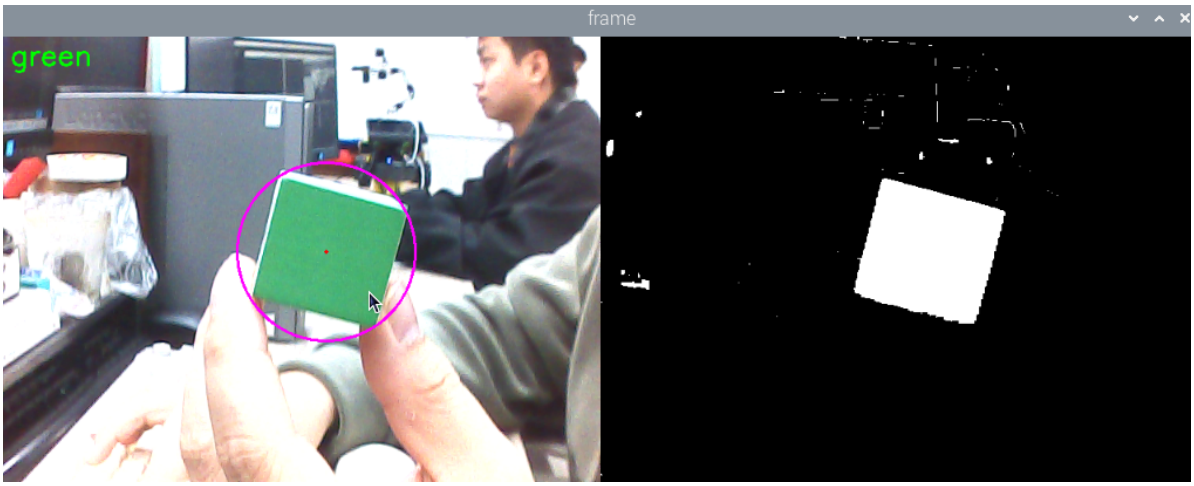
After the program runs, the robotic arm will lift up. Hold a color block and place it within the camera's field of view. Press keys to select the color to track (r-red, g-green, b-blue, y-yellow). If the color recognition effect is not good, you can press the 'c' key to enter color calibration mode. Complete color block calibration by selecting colors within the color block area, and finally press the spacebar. The robotic arm will adjust its posture to keep the color block's center coordinates within the center of the image. Slowly move the color block, and the robotic arm will follow the movement.

2. Function Startup

After entering the docker container, enter the following commands in the docker terminal:

```
# Start USB camera node and inverse kinematics node
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start color block tracking program node
ros2 run dofbot_follow color_follow_2D
```

After the program runs successfully, the robotic arm will move to its initial posture. Hold a color block and place it within the robotic arm's camera field of view. Press keys to select the color to track (r-red, g-green, b-blue, y-yellow). If the held color block cannot be recognized or the recognition effect is poor at this time, press the 'c' key to enter color calibration mode. Press the mouse to select colors within a certain area of the color block, release to complete color calibration, and finally press the spacebar. The robotic arm will start tracking the color block, adjusting its posture to keep the color block's center coordinates within the center of the image, as shown in the figure below:



3. Code Analysis

After entering the docker container, the source code path is located at:

```
/root/dofbot_ws/src/dofbot_follow/dofbot_follow/color_follow_2D.py
```

Import necessary library files:

```
import cv2
import os
import numpy as np
from sensor_msgs.msg import Image
from cv_bridge import CvBridge
import cv2 as cv
# Defined color recognition library, source code located at
/root/dofbot_ws/src/dofbot_sorting/dofbot_sorting/color_common.py
from dofbot_sorting.color_common import *
import time
from rclpy.node import Node
import rclpy
import threading
from Arm_Lib import Arm_Device
# Custom PID calculation library, located at
/root/dofbot_ws/src/dofbot_follow/dofbot_follow/PID.py
import dofbot_follow.PID as PID
```

Create robotic arm control object and control the robotic arm to move to gesture recognition posture:

```
self.Arm = Arm_Device()
self.init_joints = [90, 150, 0, 20, 90, 0]
self.Arm.Arm_serial_servo_write6_array(self.init_joints,2000)
```

Initialize PID:

```
self.xservo_pid = PID.PositionalPID(0.5, 0.02, 0.1)
self.yservo_pid = PID.PositionalPID(0.5, 0.02, 0.1)
```

Create subscriber to image topic:

```
self.subscription =
self.create_subscription(Image,'/image_raw',self.ImageCallback,1)
```

Image topic callback function ImageCallback:

```
def ImageCallback(self,color_frame):
    # Process received image topic messages and convert to image
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame,'rgb8')
    rgb_image = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2BGR)
    result_image = np.copy(rgb_image)
    key = cv2.waitKey(10)& 0xFF
    # Pass image and key detection results to process function
    result_frame, binary = self.process(rgb_image,key)
    # Create thread to display image
    show_frame = threading.Thread(target=self.img_out, args=
(result_frame,binary,))
    show_frame.start()
    show_frame.join()
    # Detect key press. If spacebar is pressed (spacebar value is 32), modify
self.start_track value to True, indicating color block tracking can start
```

```
    if key == 32:
        self.start_track = True
    # If color block is detected and the radius of its circumscribed circle is
greater than 30, start determining the color block's center point coordinates
    if self.cx!=0 and self.cy!=0 and self.circle_r>30:
        center_x = self.cx
        center_y = self.cy
        # If the color block's center coordinates x or y are not within the
center range and self.start_track value is True, execute XY_track function with
the current color block center coordinates as parameters
        if ((abs(center_x-320) >30 or abs(center_y-240)>30)) and
self.start_track == True:
            self.XY_track(center_x,center_y)
            print("Tracking")
            print("-------------------------------------" )
```